Acciones semánticas de los diagramas Talos

Manual de correspondencia entre la Fe de errata y el analizador matricial

Este manual permite ubicar una acción en un diagrama, seguir su producción numérica y comprobar su efecto en las pilas y cuádruplos. También permite partir del código y recuperar el recorrido del diagrama. Incluye los 29 diagramas sintácticos de las páginas 7 a 12, las familias léxicas de las páginas 4 y 5 y CASE como ampliación de Autómatas II.

Estado verificado: 130 producciones activas y 58 acciones activas ejercitadas; 566 verificaciones reales sin fallos. El verificador independiente contrastó 1035 casos simbólicos sin discrepancias. La tabla II tiene 60 filas y 66 columnas. La posición 131 permite llamadas como estatutos; 132 y 133 aceptan = o : antes del tipo de retorno y de cada grupo de parámetros, conservando los índices anteriores.

Cómo leer una ficha

La imagen reproduce el diagrama original. Debajo se coloca su recorrido mediante producciones anotadas: {NOMBRE:3000} es una acción que se ejecuta exactamente en esa posición. El número es el código real del C++. La forma factorizada puede distribuir un diagrama en varias producciones; la ficha las reúne para no confundir un auxiliar con un diagrama distinto.

PilaO = operandos; PilaTipos = tipos; POper = operadores; PilaSaltos = saltos; AVAIL = temporales libres. PUSH agrega al tope; POP extrae del tope. Para una operación binaria, el primer POP obtiene el derecho y el segundo el izquierdo. CONT es el número del próximo cuádruplo, desde 1.

La producción P se almacena en la fila P-1 de MProducciones. La columna 0 guarda longitud; la primera entrada está en columna 1. El TSV incluye fila y columna físicas: MProducciones[P-1][columna].

Correspondencia reversible significa quitar las acciones y recuperar exactamente la gramática, y conservar el enlace de cada acción a una producción y a un diagrama. No significa invertir una asignación ni deshacer la ejecución del programa. El deslizador reconstruye los eventos de generación. Saltos y rellenos conserva el pendiente y su destino; Fondos falsos conserva PUSH y POP aun después de retirar la marca. La tercera vista recorre la ejecución real con PC, condición V/F y destino siguiente, y dibuja las conexiones de los cuádruplos.

Fuentes y decisiones de adaptación

Fuente sintáctica: Proyecto Lenguajes y Automatas Talos Fe de errata (1).pdf, páginas PDF 4 a 12. Fuente de acciones: Material Lenguajes y Automatas 2.1.pdf, generación de código y módulos, páginas PDF 30 a 48; módulo de procedimientos en las páginas impresas 48 y 49. Las acciones de funciones con valor de retorno son la adaptación a Talos que solicita ese material.

Se conserva int y end_function como escritura canónica. El listado léxico menciona endfunction y endlib, pero los diagramas sintácticos usan end_function y no contienen una construcción endlib. No se inventa un estatuto para ese token. No se agrega then a IF. Las llamadas reciben ids como argumentos; el diagrama no permite EXPR en esa posición.

Índice de diagramas y producciones

01 PROGRAM | Fe p. 7 | P1, P2, P3, P4, P5

02 DECLARA_LIBRARY | Fe p. 8 | P7, P8

03 DECLARACONST | Fe p. 8 | P9

04 DECLARA_CLASS | Fe p. 8 | P6, P127

05 DECLARAVAR | Fe p. 8 | P2, P11, P12, P13, P14, P15, P125, P126

06 TIPO_DE_DATO | Fe p. 9 | P16, P17, P18, P19, P20, P21

07 VALOR_CONSTANTE | Fe p. 9 | P22, P23, P24, P25, P26, P128, P129

08 DECLARA_FUNCTION | Fe p. 9 | P34, P132, P133

09 PARAMETROS | Fe p. 9 | P27, P28, P29, P30, P31, P32, P33, P132, P133

10 ESTATUTOS | Fe p. 10 | P36, P37, P38, P39, P40, P41, P42, P43, P44, P45, P46, P47, P48, P107, P131

11 EST_ASIG | Fe p. 10 | P45, P46, P49

12 EST_READ | Fe p. 10 | P50, P51, P52, P53

13 EST_WRITE | Fe p. 10 | P54, P55, P56, P57

14 PREINCUNARIO | Fe p. 10 | P58, P59

15 POSTINCUNARIO | Fe p. 10 | P45, P47, P60, P61

16 EST_DO | Fe p. 11 | P62

17 EST_IF | Fe p. 11 | P63, P64, P65, P66, P67

18 EST_WHILE | Fe p. 11 | P68

19 EST_FOR | Fe p. 11 | P69

20 EST_RETURN | Fe p. 11 | P70

21 EXPR | Fe p. 12 | P71, P72, P73

22 EXPR2 | Fe p. 12 | P74, P75, P76

23 EXPR3 | Fe p. 12 | P77, P78

24 EXPR4 | Fe p. 12 | P79, P80, P81

25 OPREL | Fe p. 12 | P91, P92, P93, P94, P95, P96

26 EXPR5 | Fe p. 12 | P82, P83, P84, P85

27 TERM | Fe p. 12 | P86, P87, P88, P89, P90, P106

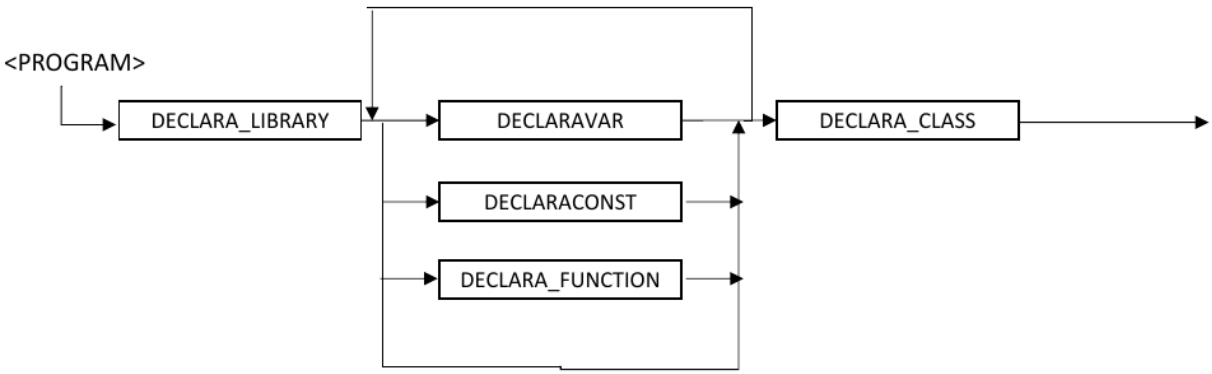
28 FACT | Fe p. 12 | P97, P98, P99, P130

29 LLAMADA_F | Fe p. 12 | P100, P101, P102, P103, P104, P105

Después de las fichas: autómatas léxicos, CASE, ejemplo de IF con relleno de saltos, ejemplo de funciones y protocolo de verificación. Las posiciones 10, 35 y 48 se conservan retiradas y no se ejecutan.

01 PROGRAM

Diagrama original de la Fe de errata, página PDF 7.



PROGRAM2 factoriza la repetición de declaraciones. El programa principal empieza después de las definiciones; los goto de cada función evitan ejecutar sus cuerpos al cargar el programa. Al llegar a EOF se comprueban pilas y saltos.

Recorrido con las acciones colocadas

```
P1 PROGRAM → DECLARA_LIBRARY PROGRAM2 DECLARA_CLASS
P2 PROGRAM2 → {DECL_BEGIN:3000} def IDCOF TIPOSEP TIPO ; {DECL_END:3003} PROGRAM2
P3 PROGRAM2 → DECLARACONST PROGRAM2
P4 PROGRAM2 → DECLARA_FUNCTION PROGRAM2
P5 PROGRAM2 → ε
```

Efecto de las acciones

A3000 DECL_BEGIN Vaciar nombres pendientes y tipo de declaración. No modificar símbolos ya declarados.

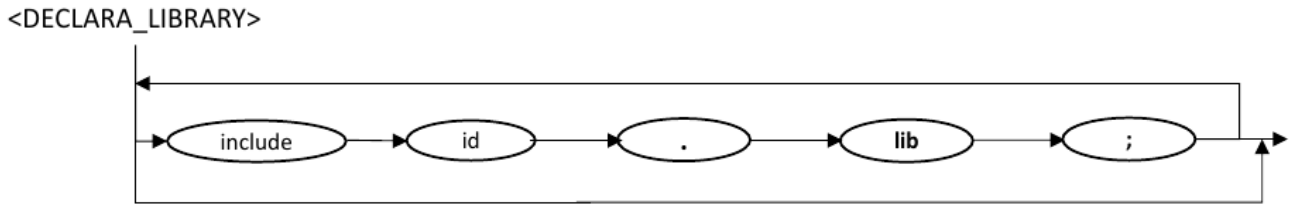
A3003 DECL_END Para cada id pendiente, rechazar duplicado en el ámbito actual; reservar AP, RAM virtual y tipo.

Ejemplo de aplicación

```
include matematicas.lib; def a of int; class a=1; endclass
```

02 DECLARA_LIBRARY

Diagrama original de la Fe de errata, página PDF 8.



LIB_NAME registra el nombre lógico de la biblioteca. No carga DLL ni traduce el archivo .lib: la Fe de errata sólo define el reconocimiento de include id . lib ;. El ejecutor resuelve las operaciones del lenguaje implementado.

Recorrido con las acciones colocadas

`P7 DECLARA_LIBRARY → include id {LIB_NAME:3059} . lib ; DECLARA_LIBRARY`

`P8 DECLARA_LIBRARY → ε`

Efecto de las acciones

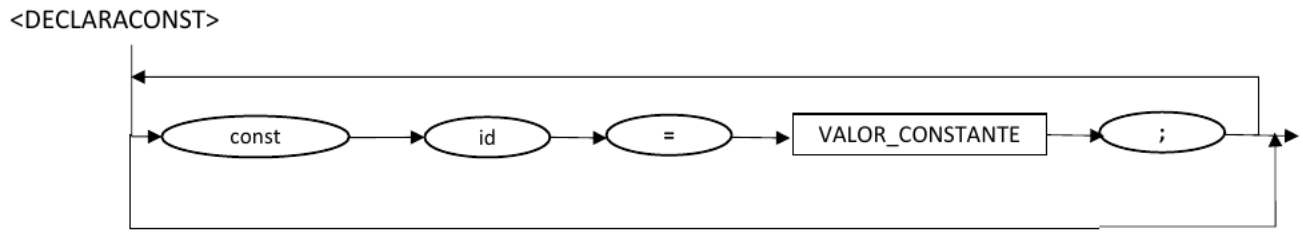
A3059 LIB_NAME Registrar id en el arreglo lógico de bibliotecas, evitando duplicados. No ejecuta código externo.

Ejemplo de aplicación

```
include matematicas.lib;
```

03 DECLARACONST

Diagrama original de la Fe de errata, página PDF 8.



El valor se registra en la tabla de constantes; CONST_END crea un símbolo nombrado de sólo lectura y copia su valor inicial. La posición 10 está retirada: no se utiliza para reconocer una declaración vacía.

Recorrido con las acciones colocadas

```
P9 DECLARACONST → const id {CONST_NAME:3004} = VALOR {CONST_END:3005} ;
```

Efecto de las acciones

A3004 CONST_NAME Guardar el nombre situado después de const para completar la declaración.

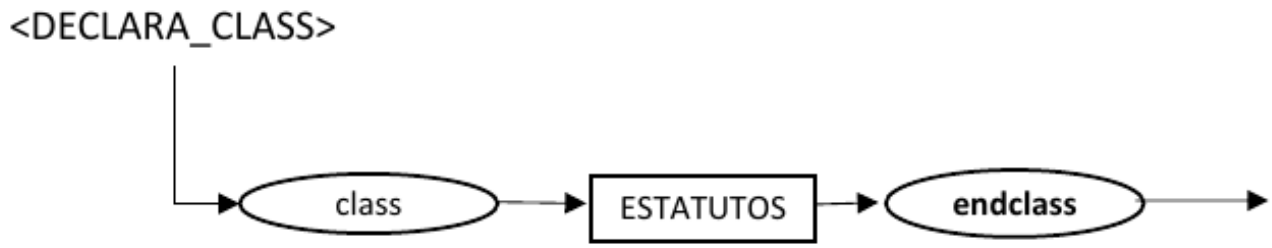
A3005 CONST_END POP operando y tipo del literal; declarar símbolo de sólo lectura con ese valor inicial.

Ejemplo de aplicación

```
const limite=254;
```

04 DECLARA_CLASS

Diagrama original de la Fe de errata, página PDF 8.



class/endclass delimita el cuerpo principal. No hay instancia de objeto ni herencia en este diagrama. La alternativa 127 permite el cuerpo sin envoltura usado en los apuntes de II; ambas rutas comparten ESTATUTOS.

Recorrido con las acciones colocadas

`P6 DECLARA_CLASS → class ESTATUTOS endclass`

`P127 DECLARA_CLASS → ESTATUTOS`

Efecto de las acciones

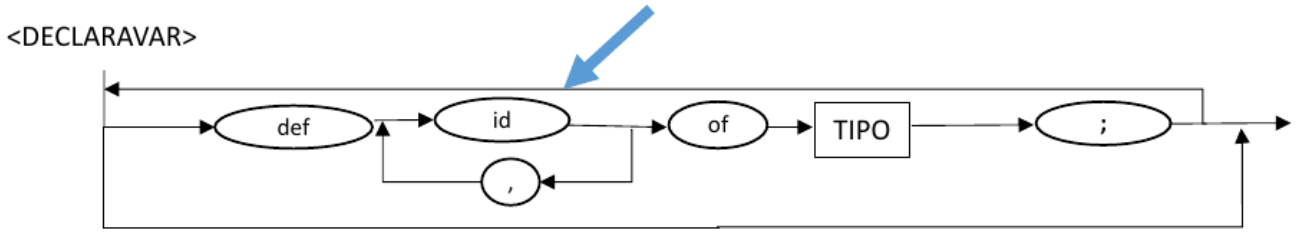
La regla reconoce estructura y delega. No realiza PUSH/POP ni genera un cuádruplo adicional. La validación global al llegar a EOF exige pilas cerradas y saltos resueltos.

Ejemplo de aplicación

```
class write(1); endclass
```

05 DECLARAVAR

Diagrama original de la Fe de errata, página PDF 8.



DECL_BEGIN vacía la lista pendiente; cada id agrega un nombre; TYPE fija el tipo; DECL_END declara todo el grupo. En la base, PROGRAM2 contiene el primer def y DECLARAVAR las declaraciones locales. TIPOSEP acepta of o dos puntos.

Recorrido con las acciones colocadas

```
P2 PROGRAM2 → {DECL_BEGIN:3000} def IDCOF TIPOSEP TIPO ; {DECL_END:3003} PROGRAM2
P11 IDCOF → id {DECL_ID:3001} IDCOF'
P12 IDCOF' → , IDCOF
P13 IDCOF' → ε
P14 DECLARAVAR → {DECL_BEGIN:3000} def IDCOF TIPOSEP TIPO ; {DECL_END:3003} DECLARAVAR
P15 DECLARAVAR → ε
P125 TIPOSEP → of
P126 TIPOSEP → :
```

Efecto de las acciones

A3000 DECL_BEGIN Vaciar nombres pendientes y tipo de declaración. No modificar símbolos ya declarados.

A3001 DECL_ID Agregar id al arreglo pendiente, comprobando capacidad.

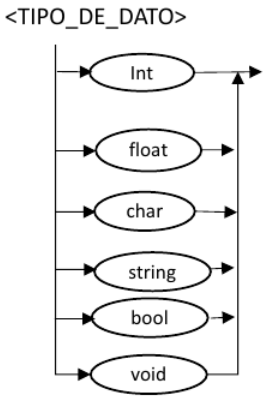
A3003 DECL_END Para cada id pendiente, rechazar duplicado en el ámbito actual; reservar AP, RAM virtual y tipo.

Ejemplo de aplicación

```
def a,b of int;
```

06 TIPO_DE_DATO

Diagrama original de la Fe de errata, página PDF 9.



Nombre del no terminal en C++: TIPO. TYPE traduce el token a 1=int, 2=float, 3=char, 4=string, 5=bool o 6=void. void sólo es retorno de función; no admite variable ni parámetro. El Int del dibujo se interpreta como int, reservado canónico.

Recorrido con las acciones colocadas

```
P16 TIPO → int {TYPE:3002}
P17 TIPO → float {TYPE:3002}
P18 TIPO → char {TYPE:3002}
P19 TIPO → string {TYPE:3002}
P20 TIPO → bool {TYPE:3002}
P21 TIPO → void {TYPE:3002}
```

Efecto de las acciones

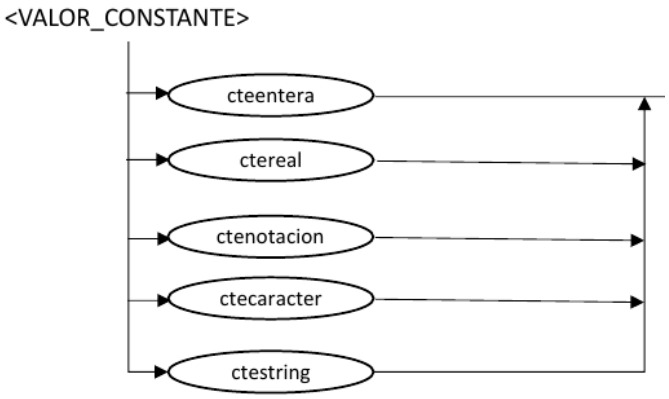
A3002 TYPE Traducir el código del token al índice de tipo de MCubo y MAsignacion.

Ejemplo de aplicación

```
def radio of float;
```

07 VALOR_CONSTANTE

Diagrama original de la Fe de errata, página PDF 9.



Nombre en C++: VALOR. VALUE busca o registra el literal por tipo y valor, y apila AP y tipo. Entero, real y notación científica conservan su clasificación léxica; real y científica se operan como float. true/false son la extensión booleana de II.

Recorrido con las acciones colocadas

```
P22 VALOR → cteentera {VALUE:3006}
P23 VALOR → ctereal {VALUE:3006}
P24 VALOR → ctenotacion {VALUE:3006}
P25 VALOR → ctecaracter {VALUE:3006}
P26 VALOR → ctestring {VALUE:3006}
P128 VALOR → true {VALUE:3006}
P129 VALOR → false {VALUE:3006}
```

Efecto de las acciones

A3006 VALUE Buscar o insertar literal en MConstantes; PUSH(AP) en operandos y PUSH(tipo) en tipos.

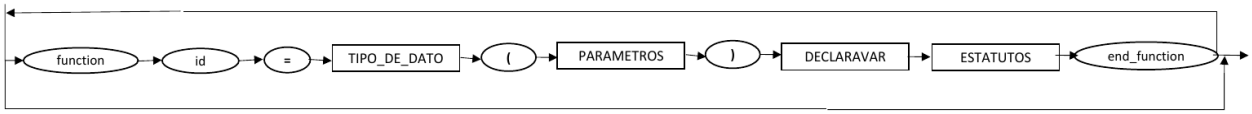
Ejemplo de aplicación

```
write(254,0.25,1.25e2,'a',"hola");
```

08 DECLARA_FUNCTION

Diagrama original de la Fe de errata, página PDF 9.

<DECLARA_FUNCTION>



El salto inicial reserva el cuerpo. Se registra firma y ámbito antes de compilar estatutos, lo que permite recursión directa. FUNC_END comprueba return en todos los caminos alcanzables para funciones no void y rellena el salto que evita ejecutar la definición.

Recorrido con las acciones colocadas

```
P34 DECLARA_FUNCTION → function id {FUNC_NAME:3044} TIPOFUNCSEP TIPO {FUNC_TYPE:3045} ( PARAMETROS )
{FUNC_PARAMS_END:3048} DECLARAVAR {FUNC_BODY:3049} ESTATUTOS end_function {FUNC_END:3050}
```

```
P132 TIPOFUNCSEP → =
```

```
P133 TIPOFUNCSEP → :
```

Efecto de las acciones

A3044 FUNC_NAME Registrar nombre único, crear ámbito f+1 y emitir goto pendiente para evitar ejecutar la definición.

A3045 FUNC_TYPE Guardar el índice de tipo de retorno en MFunciones.

A3048 FUNC_PARAMS_END Verificar que no queden nombres formales sin tipo.

A3049 FUNC_BODY Guardar CONT como entrada de función y contar parámetros y variables locales.

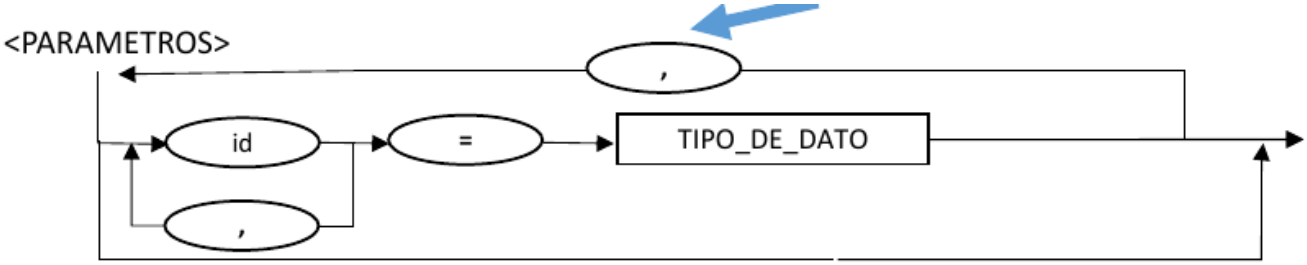
A3050 FUNC_END Emitir endfunc; recorrer flujo por índices para detectar salida sin return en no void; rellenar salto inicial y volver al ámbito global.

Ejemplo de aplicación

```
function doble=int(x=int) return x*2; end_function
```

09 PARAMETROS

Diagrama original de la Fe de errata, página PDF 9.



Un grupo a,b=int o a,b:int declara dos parámetros int. TIPOFUNCSEP es la ampliación explícita que permite : además del = original, tanto en retorno como en parámetros. Cada PARAM_TYPE consume los nombres acumulados y agrega filas función/símbolo/tipo a MParametros. Los argumentos de Talos son por valor; no se añadió paso por referencia ni sintaxis ajena al dibujo.

Recorrido con las acciones colocadas

```
P27 IDCIGUAL → id {PARAM_ID:3046} IDCIGUAL'
P28 IDCIGUAL' → , IDCIGUAL
P29 IDCIGUAL' → ε
P30 PARAMETROS → IDCIGUAL TIPOFUNCSEP TIPO {PARAM_TYPE:3047} PARAMETROS'
P31 PARAMETROS → ε
P32 PARAMETROS' → , IDCIGUAL TIPOFUNCSEP TIPO {PARAM_TYPE:3047} PARAMETROS'
P33 PARAMETROS' → ε
P132 TIPOFUNCSEP → =
P133 TIPOFUNCSEP → :
```

Efecto de las acciones

A3046 PARAM_ID Agregar el nombre de parámetro a pendientes del grupo actual.

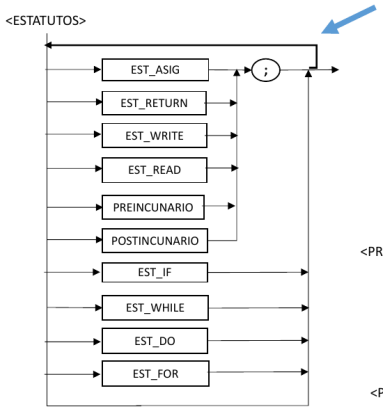
A3047 PARAM_TYPE Declarar todos los ids del grupo en el ámbito local; agregar filas de MParametros; incrementar cantidad formal.

Ejemplo de aplicación

```
function suma=int(a,b=int,c=float) return a+b; end_function
```

10 ESTATUTOS

Diagrama original de la Fe de errata, página PDF 10.



El id común de asignación y postincremento se factoriza en ESTATUTOS prima. STMT_NAME conserva su referencia; la producción siguiente decide asignación, ++, -- o llamada. Se añadió la llamada como estatuto para utilizar funciones void. El punto y coma pertenece a este nivel.

Recorrido con las acciones colocadas

```
P36 ESTATUTOS → EST_RETURN ; ESTATUTOS
P37 ESTATUTOS → EST_WRITE ; ESTATUTOS
P38 ESTATUTOS → EST_READ ; ESTATUTOS
P39 ESTATUTOS → PREINCUNARIO ; ESTATUTOS
P40 ESTATUTOS → EST_IF ESTATUTOS
P41 ESTATUTOS → EST_WHILE ESTATUTOS
P42 ESTATUTOS → EST_DO ESTATUTOS
P43 ESTATUTOS → EST_FOR ESTATUTOS
P44 ESTATUTOS → ε
P45 ESTATUTOS → id {STMT_NAME:3058} ESTATUTOS'
P46 ESTATUTOS' → EST_ASIG ; ESTATUTOS
P47 ESTATUTOS' → POSTINCUNARIO ; ESTATUTOS
P107 ESTATUTOS → CASE ESTATUTOS
P131 ESTATUTOS' → {CALL_BEGIN:3053} ( IDCC ) {CALL_STMT_END:3056} ; ESTATUTOS
```

Efecto de las acciones

A3053 CALL_BEGIN POP referencia y tipo; exigir función declarada; crear fila de llamada, emitir era y PUSH de fondo falso.

A3056 CALL_STMT_END Comprobar firma y cerrar llamada; emitir gosub; si hay valor descartado, liberar su temporal. Permite void.

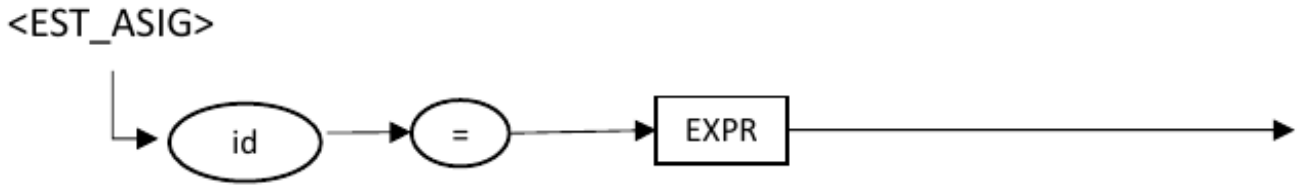
A3058 STMT_NAME Resolver id como destino o referencia de función; la siguiente producción decide su uso.

Ejemplo de aplicación

```
a=2; a++; write(a);
```

11 EST_ASIG

Diagrama original de la Fe de errata, página PDF 10.



El id se reconoce en ESTATUTOS y queda en la pila debajo del resultado de EXPR. ASSIGN saca primero origen y luego destino; consulta MAsignacion y prohíbe escribir constantes. El orden es (=, origen, vacío, destino).

Recorrido con las acciones colocadas

```
P45 ESTATUTOS → id {STMT_NAME:3058} ESTATUTOS'
P46 ESTATUTOS' → EST_ASIG ; ESTATUTOS
P49 EST_ASIG → = EXPR {ASSIGN:3008}
```

Efecto de las acciones

A3008 ASSIGN POP origen y tipo; POP destino y tipo; consultar MAsignacion; emitir (=, origen, vacío, destino); devolver temporal origen a AVAIL.

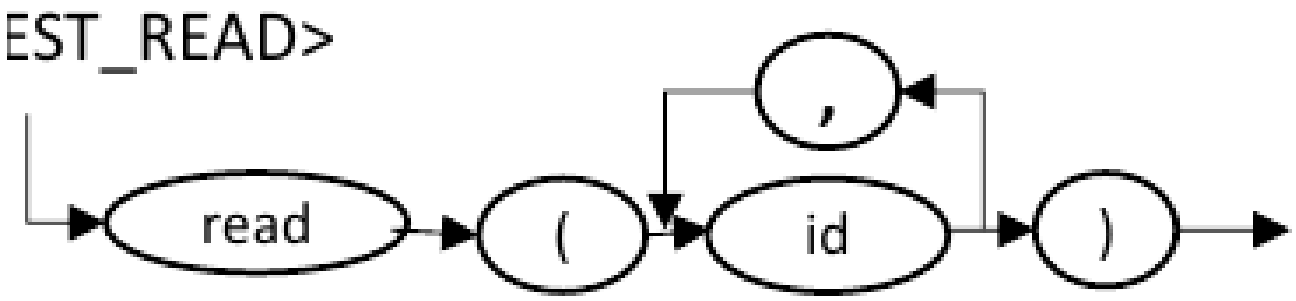
A3058 STMT_NAME Resolver id como destino o referencia de función; la siguiente producción decide su uso.

Ejemplo de aplicación

```
a=b*2;
```

12 EST_READ

Diagrama original de la Fe de errata, página PDF 10.



READ verifica cada id y emite un in por elemento. El tipo de la entrada concreta se comprueba al ejecutar; al generar no hay un dato del usuario que sacar de la pila. La entrada no puede escribir en una constante.

Recorrido con las acciones colocadas

```
P50 IDCREAD → id {READ:3009} IDCREAD'
P51 IDCREAD' → , IDCREAD
P52 IDCREAD' → ε
P53 EST_READ → read ( IDCREAD )
```

Efecto de las acciones

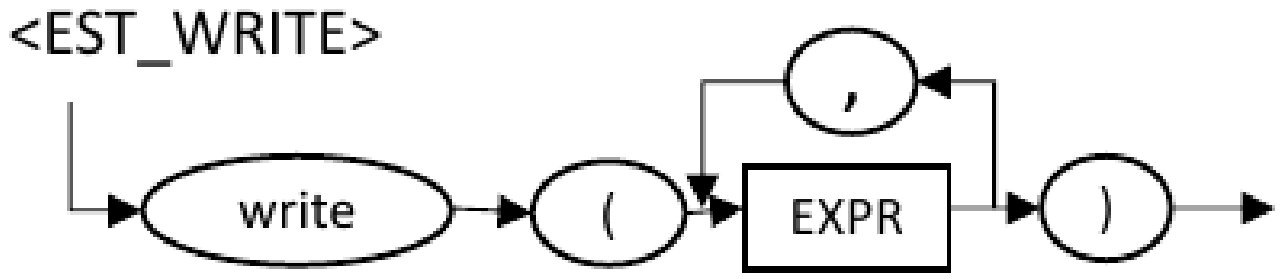
A3009 READ Buscar id; comprobar variable modificable; emitir (in, vacío, vacío, AP). El dato se valida al ejecutar.

Ejemplo de aplicación

```
read(a,b);
```

13 EST_WRITE

Diagrama original de la Fe de errata, página PDF 10.



Cada EXPR se resuelve y OUT consume un operando y su tipo. La lista emite un out por expresión y libera temporales. Este ejecutor escribe un salto de línea por out; la lista no se concatena automáticamente.

Recorrido con las acciones colocadas

```
P54 WRITE2 → EXPR {OUT:3010} WRITE2'
P55 WRITE2' → , WRITE2
P56 WRITE2' → ε
P57 EST_WRITE → write ( WRITE2 )
```

Efecto de las acciones

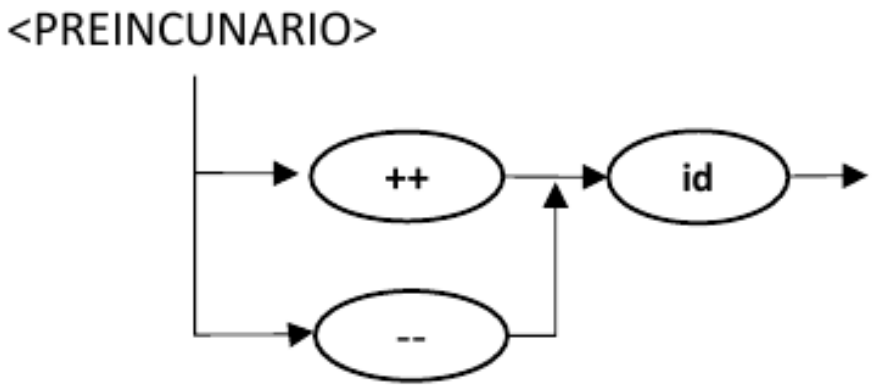
A3010 OUT POP operando y tipo; emitir (out, vacío, vacío, AP); liberar temporal consumido.

Ejemplo de aplicación

```
write("Resultado",a+1);
```

14 PREINCUNARIO

Diagrama original de la Fe de errata, página PDF 10.



INC/DEC resuelven id después del signo y verifican que sea una variable numérica modificable. Emiten (+ o -, id, constante 1, id). En Talos son estatutos; no se usan como valores dentro de expresiones.

Recorrido con las acciones colocadas

`P58 PREINCUNARIO → ++ id {INC:3017}`

`P59 PREINCUNARIO → -- id {DEC:3018}`

Efecto de las acciones

A3017 INC Resolver id; exigir numérico modificable; emitir (+, id, 1, id).

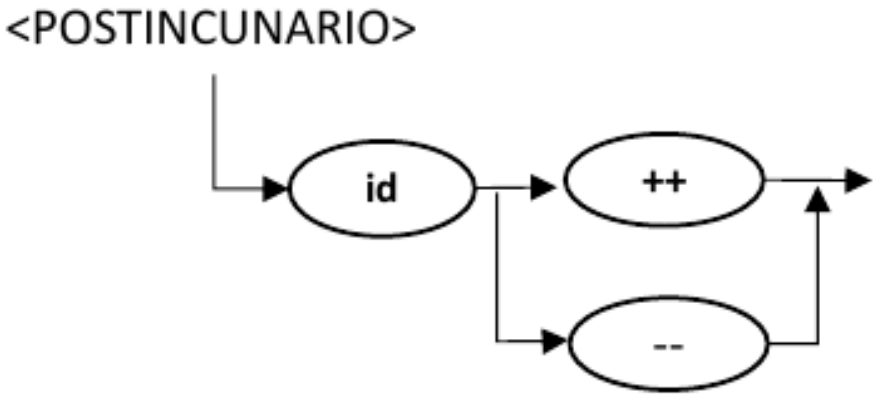
A3018 DEC Resolver id; exigir numérico modificable; emitir (-, id, 1, id).

Ejemplo de aplicación

```
++a; --a;
```

15 POSTINCUNARIO

Diagrama original de la Fe de errata, página PDF 10.



El id ya quedó apilado al factorizar ESTATUTOS. POSTINC/POSTDEC lo consumen y emiten la actualización in situ. No hace falta producir un temporal con el valor anterior porque el diagrama describe un estatuto.

Recorrido con las acciones colocadas

```
P45 ESTATUTOS → id {STMT_NAME:3058} ESTATUTOS'
P47 ESTATUTOS' → POSTINCUNARIO ; ESTATUTOS
P60 POSTINCUNARIO → ++ {POSTINC:3019}
P61 POSTINCUNARIO → -- {POSTDEC:3020}
```

Efecto de las acciones

A3019 POSTINC POP id y tipo previamente reconocidos; validar; emitir (+, id, 1, id).

A3020 POSTDEC POP id y tipo previamente reconocidos; validar; emitir (-, id, 1, id).

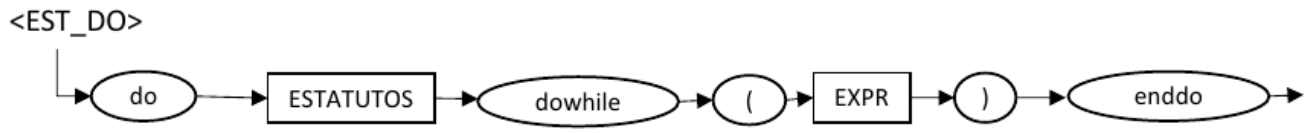
A3058 STMT_NAME Resolver id como destino o referencia de función; la siguiente producción decide su uso.

Ejemplo de aplicación

```
a++; a--;
```

16 EST_DO

Diagrama original de la Fe de errata, página PDF 11.



DO_BEGIN guarda CONT antes del cuerpo. DO_END exige condición bool y emite gotov hacia ese inicio. Se repite mientras sea verdadera: no es la operación repeat-until de otros lenguajes.

Recorrido con las acciones colocadas

```
P62 EST_DO → do {DO_BEGIN:3036} ESTATUTOS dowhile ( EXPR ) enddo {DO_END:3037}
```

Efecto de las acciones

A3036 DO_BEGIN Crear marca de ciclo y guardar CONT, anterior al primer estatuto del cuerpo.

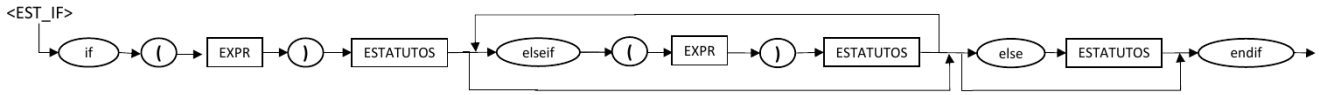
A3037 DO_END POP bool; emitir gotov hacia el inicio del cuerpo; quitar marca y contexto.

Ejemplo de aplicación

```
do a++; dowhile(a<3) enddo
```

17 EST_IF

Diagrama original de la Fe de errata, página PDF 11.



IF1 representa cada elseif y su repetición; IF2 representa else opcional. IF_NEXT emite salida de la rama anterior y rellena su falso hacia la condición siguiente o else. IF_END rellena todas las salidas hasta la marca de este IF. Talos no contiene la palabra then.

Recorrido con las acciones colocadas

```
P63 IF1 → {IF_NEXT:3023} elseif ( EXPR ) {IF_TEST:3022} ESTATUTOS IF1
P64 IF1 → ε
P65 IF2 → {IF_NEXT:3023} else ESTATUTOS
P66 IF2 → ε
P67 EST_IF → {IF_BEGIN:3021} if ( EXPR ) {IF_TEST:3022} ESTATUTOS IF1 IF2 endif {IF_END:3024}
```

Efecto de las acciones

A3021 IF_BEGIN Crear fila de contexto IF y PUSH de una marca propia en saltos.

A3022 IF_TEST POP condición y tipo bool; emitir gotofalso pendiente; PUSH de su número en saltos; liberar condición temporal.

A3023 IF_NEXT Emitir goto pendiente para saltar las ramas restantes; POP del falso anterior y rellenarlo con CONT; PUSH de la nueva salida.

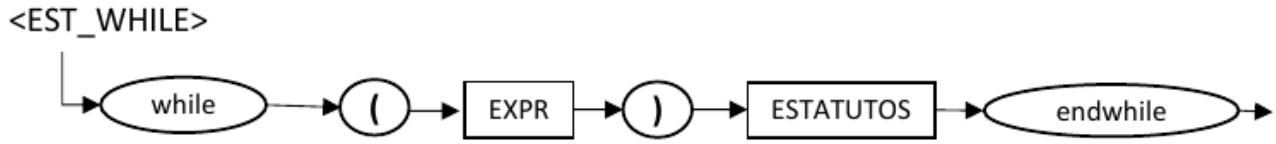
A3024 IF_END Rellenar salidas y falso pendiente con CONT hasta la marca de este IF; retirar marca y contexto.

Ejemplo de aplicación

```
if(a<0) write(0); elseif(a==0) write(1); else write(2); endif
```

18 EST_WHILE

Diagrama original de la Fe de errata, página PDF 11.



WHILE_BEGIN guarda la dirección de la condición. WHILE_TEST consume bool y reserva gotofalso. WHILE_END emite goto al inicio y rellena el falso con la siguiente instrucción. Las marcas separan ciclos anidados.

Recorrido con las acciones colocadas

`P68 EST_WHILE → {WHILE_BEGIN:3033} while ( EXPR ) {WHILE_TEST:3034} ESTATUTOS endwhile {WHILE_END:3035}`

Efecto de las acciones

A3033 WHILE_BEGIN Crear marca de ciclo; guardar CONT como dirección de la condición.

A3034 WHILE_TEST POP bool; emitir gotofalso pendiente y apilar número; liberar temporal.

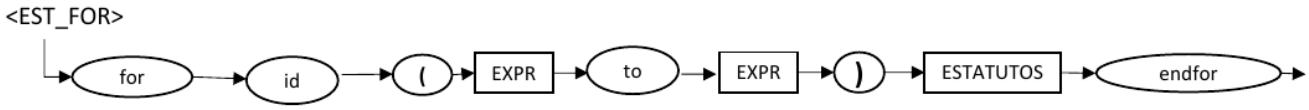
A3035 WHILE_END Emitir goto hacia la condición; rellenar gotofalso con CONT; quitar marca y contexto.

Ejemplo de aplicación

```
while(a<3) a++; endwhile
```

19 EST_FOR

Diagrama original de la Fe de errata, página PDF 11.



FOR_BEGIN exige contador int modificable. Se asigna el inicio; el límite se evalúa una vez y se conserva en un temporal reservado. Se prueba contador <= límite. FOR_END incrementa, vuelve a probar y libera el límite.

Recorrido con las acciones colocadas

```
P69 EST_FOR → for id {FOR_BEGIN:3038} ( EXPR {FOR_INIT:3039} to EXPR {FOR_TEST:3040} ) ESTATUTOS endfor {FOR_END:3041}
```

Efecto de las acciones

A3038 FOR_BEGIN Resolver contador int modificable y crear contexto FOR.

A3039 FOR_INIT POP inicio int; emitir asignación al contador y liberar temporal consumido.

A3040 FOR_TEST POP límite int; copiarlo a temporal reservado; guardar inicio de prueba; emitir <= y gotofalso pendiente.

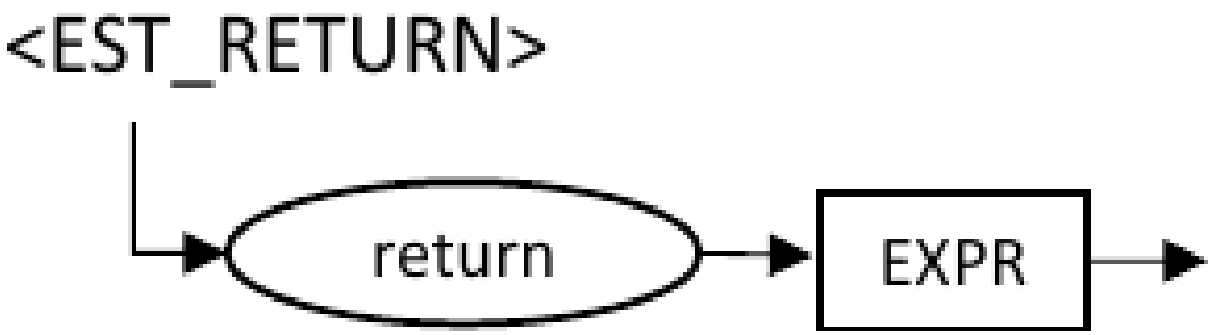
A3041 FOR_END Incrementar contador; emitir goto a prueba; rellenar salida; devolver límite reservado a AVAIL.

Ejemplo de aplicación

```
for a (1 to 3) write(a); endfor
```

20 EST_RETURN

Diagrama original de la Fe de errata, página PDF 11.



RETURN_VALUE consume el valor de EXPR, valida su tipo contra la firma y emite return. Sólo se permite dentro de función no void. En ejecución copia el valor antes de restaurar la activación del llamador y salta a su dirección de retorno.

Recorrido con las acciones colocadas

`P70 EST_RETURN → return EXPR {RETURN_VALUE:3057}`

Efecto de las acciones

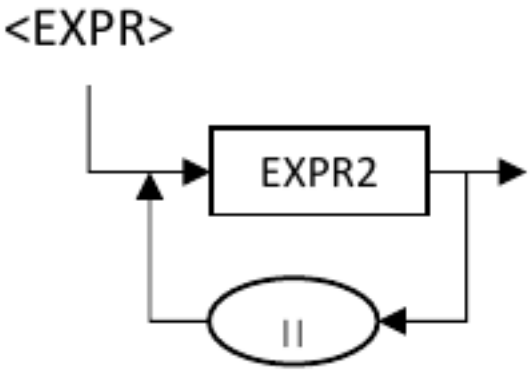
A3057 RETURN_VALUE POP valor y tipo; exigir función actual no void y compatibilidad con retorno; emitir return y liberar temporal del generador.

Ejemplo de aplicación

```
return a+1;
```

21 EXPR

Diagrama original de la Fe de errata, página PDF 12.



El nivel exterior admite OR. PUSH_OP guarda || después de reducir prioridades mayores o iguales; EXPR_END resuelve lo pendiente hasta el fondo falso. La reducción toma derecho antes de izquierdo y apila temporal y tipo resultante.

Recorrido con las acciones colocadas

```
P71 EXPR → EXPR2 EXPR' {EXPR_END:3012}
P72 EXPR' → || {PUSH_OP:3011} EXPR
P73 EXPR' → ε
```

Efecto de las acciones

A3011 PUSH_OP Mientras el tope tenga prioridad mayor o igual y no sea fondo falso, reducir. PUSH del operador reconocido.

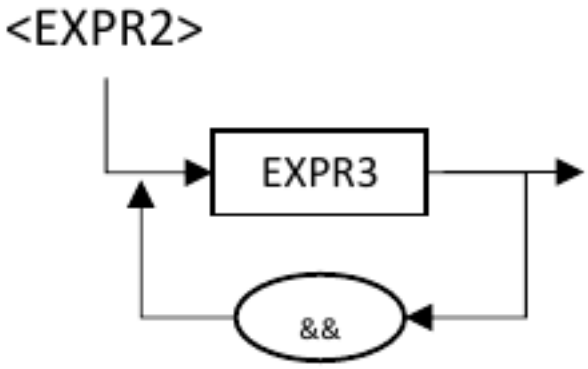
A3012 EXPR_END Reducir operadores hasta vacío o fondo falso. Cada reducción usa MCubo, toma temporal de AVAIL y apila el resultado.

Ejemplo de aplicación

```
write(true || false);
```

22 EXPR2

Diagrama original de la Fe de errata, página PDF 12.



Este nivel admite AND. PUSH_OP recibe &&. Su prioridad es mayor que || y menor que relaciones. El retorno a EXPR y el siguiente operador desencadenan la reducción; no se emiten cuádruplos adicionales por una alternativa vacía.

Recorrido con las acciones colocadas

```
P74 EXPR2 → EXPR3 EXPR2'
P75 EXPR2' → && {PUSH_OP:3011} EXPR2
P76 EXPR2' → ε
```

Efecto de las acciones

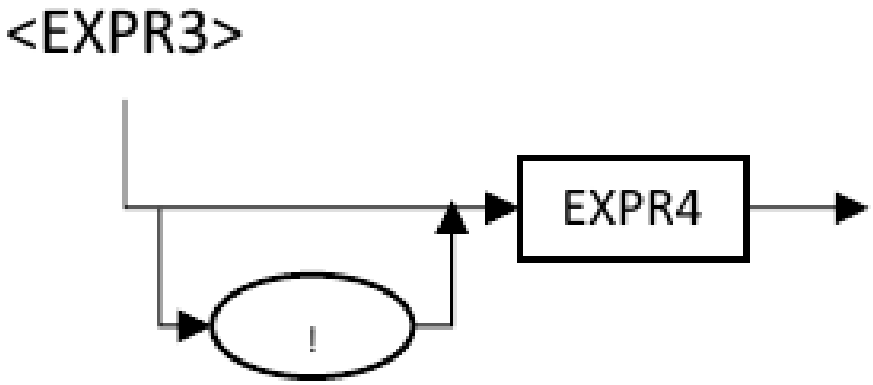
A3011 PUSH_OP Mientras el tope tenga prioridad mayor o igual y no sea fondo falso, reducir. PUSH del operador reconocido.

Ejemplo de aplicación

```
write(true && false);
```

23 EXPR3

Diagrama original de la Fe de errata, página PDF 12.



PUSH_OP recibe !, de aridad uno. Al reducir se saca sólo un operando bool y se emite (!, operando, vacío, temporal). En este diagrama NOT se aplica a EXPR4 completo; no se debe imponer la precedencia de C sin revisar el dibujo.

Recorrido con las acciones colocadas

P77 EXPR3 → EXPR4

P78 EXPR3 → ! {PUSH_OP:3011} EXPR4

Efecto de las acciones

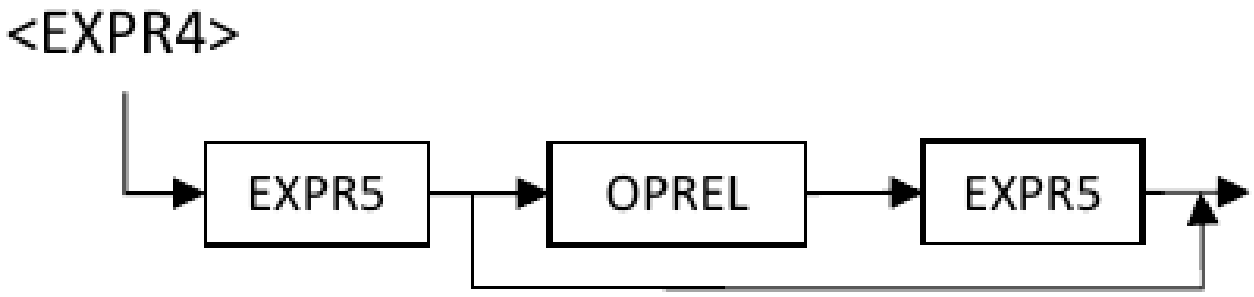
A3011 PUSH_OP Mientras el tope tenga prioridad mayor o igual y no sea fondo falso, reducir. PUSH del operador reconocido.

Ejemplo de aplicación

```
write(!(1==2));
```

24 EXPR4

Diagrama original de la Fe de errata, página PDF 12.



OPREL apila la relación entre los dos EXPR5. El dibujo admite cero o una relación, no cadenas como a<b<c. MCubo define compatibilidad y devuelve bool. EXPR41 es la cola factorizada de esta regla.

Recorrido con las acciones colocadas

`P79 EXPR4 → EXPR5 EXPR41`

`P80 EXPR41 → OPREL EXPR5`

`P81 EXPR41 → ε`

Efecto de las acciones

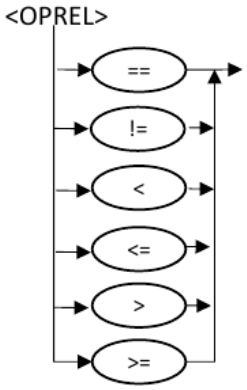
La regla reconoce estructura y delega. No realiza PUSH/POP ni genera un cuádruplo adicional. La validación global al llegar a EOF exige pilas cerradas y saltos resueltos.

Ejemplo de aplicación

```
write(2+1>=3);
```

25 OPREL

Diagrama original de la Fe de errata, página PDF 12.



Cada alternativa usa PUSH_OP con ==, !=, <, <=, > o >=. No se consulta un supuesto OPREL1: el auxiliar existente se llama EXPR41. La operación real se genera cuando la prioridad permite reducir ambos operandos.

Recorrido con las acciones colocadas

```
P91 OPREL → == {PUSH_OP:3011}
P92 OPREL → != {PUSH_OP:3011}
P93 OPREL → < {PUSH_OP:3011}
P94 OPREL → <= {PUSH_OP:3011}
P95 OPREL → > {PUSH_OP:3011}
P96 OPREL → >= {PUSH_OP:3011}
```

Efecto de las acciones

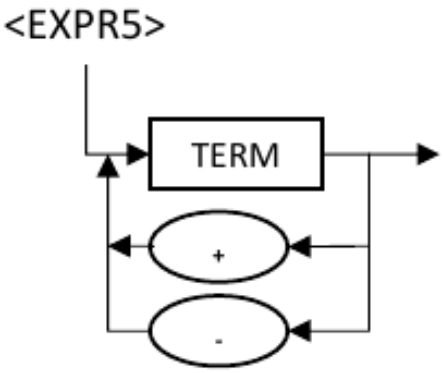
A3011 PUSH_OP Mientras el tope tenga prioridad mayor o igual y no sea fondo falso, reducir. PUSH del operador reconocido.

Ejemplo de aplicación

```
write(2!=3);
```

26 EXPR5

Diagrama original de la Fe de errata, página PDF 12.



Admite + y -. La prioridad matricial y la reducción antes de apilar otro operador del mismo nivel conservan asociación izquierda aunque la producción esté factorizada con recursión derecha.

Recorrido con las acciones colocadas

```
P82 EXPR5 → TERM EXPR5'
P83 EXPR5' → + {PUSH_OP:3011} EXPR5
P84 EXPR5' → - {PUSH_OP:3011} EXPR5
P85 EXPR5' → ε
```

Efecto de las acciones

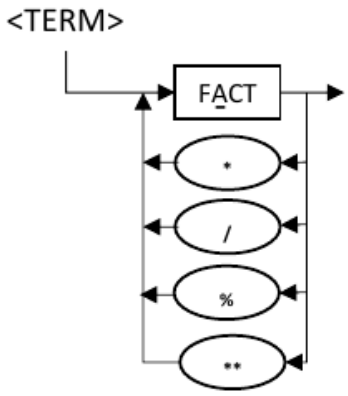
A3011 PUSH_OP Mientras el tope tenga prioridad mayor o igual y no sea fondo falso, reducir. PUSH del operador reconocido.

Ejemplo de aplicación

```
write(8-3-1); produce 4
```

27 TERM

Diagrama original de la Fe de errata, página PDF 12.



Admite *, /, % y ** según el dibujo. Comparten prioridad y asociación izquierda en Talos. / produce float; % exige int/int. La potencia no se mueve silenciosamente al nivel convencional de otros lenguajes.

Recorrido con las acciones colocadas

```
P86 TERM → FACT TERM'
P87 TERM' → * {PUSH_OP:3011} TERM
P88 TERM' → / {PUSH_OP:3011} TERM
P89 TERM' → % {PUSH_OP:3011} TERM
P90 TERM' → ε
P106 TERM' → ** {PUSH_OP:3011} TERM
```

Efecto de las acciones

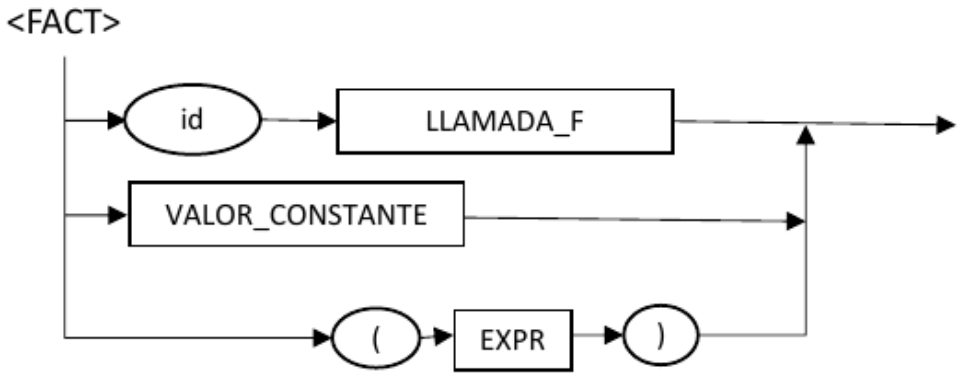
A3011 PUSH_OP Mientras el tope tenga prioridad mayor o igual y no sea fondo falso, reducir. PUSH del operador reconocido.

Ejemplo de aplicación

```
write(2*3+4, 2**3);
```

28 FACT

Diagrama original de la Fe de errata, página PDF 12.



NAME_REF distingue variable y función; LLAMADA_F decide si hay llamada o referencia simple. VALUE apila constantes. Los paréntesis colocan y retiran fondo falso. El menos unario de la foto de clase se conserva como extensión 130: NEG_PUSH y NEG_END.

Recorrido con las acciones colocadas

```
P97 FACT → id {NAME_REF:3051} LLAMADA_F
P98 FACT → VALOR
P99 FACT → ( {FF_PUSH:3013} EXPR ) {FF_POP:3014}
P130 FACT → - {NEG_PUSH:3015} FACT {NEG_END:3016}
```

Efecto de las acciones

A3013 FF_PUSH PUSH de marca de fondo falso en operadores después de abrir paréntesis.

A3014 FF_POP Comprobar marca en el tope y hacer POP; no generar un cuádruplo para el paréntesis.

A3015 NEG_PUSH PUSH de operador -u, de aridad uno y prioridad 7.

A3016 NEG_END Si el tope es -u, POP operador, un operando y su tipo; emitir negación y apilar temporal.

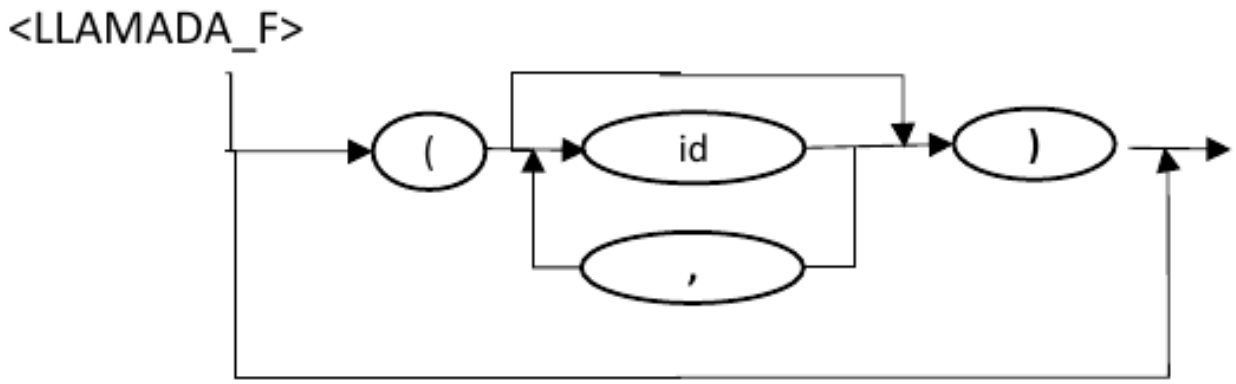
A3051 NAME_REF Buscar primero variable local y luego global; si es función, apilar referencia de función y tipo hasta decidir LLAMADA_F.

Ejemplo de aplicación

```
write(-(2+3));
```

29 LLAMADA_F

Diagrama original de la Fe de errata, página PDF 12.



La rama vacía deja la variable reconocida en FACT. La rama con paréntesis exige función. Cada argumento es un id, exactamente como dibuja la Fe de errata: no se añadieron argumentos EXPR. CALL_END deja un temporal del retorno; void sólo se usa como estatuto.

Recorrido con las acciones colocadas

```
P100 IDCC → id {CALL_ARG:3054} IDCC'
P101 IDCC → ε
P102 IDCC' → , id {CALL_ARG:3054} IDCC'
P103 IDCC' → ε
P104 LLAMADA_F → {CALL_BEGIN:3053} ( IDCC ) {CALL_END:3055}
P105 LLAMADA_F → {REF_END:3052}
```

Efecto de las acciones

A3052 REF_END En la rama vacía de LLAMADA_F exigir referencia de variable; rechazar función sin paréntesis.

A3053 CALL_BEGIN POP referencia y tipo; exigir función declarada; crear fila de llamada, emitir era y PUSH de fondo falso.

A3054 CALL_ARG Buscar id argumento; consultar formal siguiente y MAsignacion; emitir (param, AP argumento, posición, función). No alterar el argumento.

A3055 CALL_END Comprobar cantidad exacta; retirar fondo falso; reservar temporal no void; emitir gosub y PUSH del temporal y tipo de retorno.

Ejemplo de aplicación

```
resultado=doble(a);
```

Autómatas léxicos de la Fe de errata

Las páginas 4 y 5 contienen familias de reconocimiento de caracteres, no reglas para producir cuádruplos. Aquí las acciones son avanzar el apuntador, consultar M2, formar lexema, aceptar o informar error. Un token aceptado pasa después al parser; no se hace PUSH semántico de un signo por el solo hecho de leerlo.

Identificadores y reservadas

Recorrer letras, dígitos y guion bajo según M2. Al aceptar 101, consultar el arreglo de reservadas y conservar lexema, categoría y posición. Include no equivale a include: distingue mayúsculas.

Enteros reales y notación científica

102 entero; 103 real; 104 notación científica. Si falta dígito después de punto o exponente, informar error y recuperar sin perder el carácter que inicia el siguiente token. 1254. no es un real válido.

Carácter y cadena

125 y 126. Exigir delimitador de cierre y respetar el tratamiento de comillas. El char ordinal de esta implementación es un carácter ASCII; string conserva UTF8.

Asignación y relaciones

109 =; 110 ==; 111 <; 112 <=; 113 >; 114 >=; 115 !=; 116 !. Examinar el segundo carácter y devolverlo si no pertenece al operador.

Lógicos

117 && y 118 ||. Un solo & o | produce el error léxico correspondiente; ! es un operador unario válido.

Separadores

119/120 paréntesis, 121/122 corchetes, 123 punto y coma, 124 coma, 127 numeral y 134 punto. Un token reservado o separador sin producción válida puede ser correcto léxicamente y ser rechazado por sintaxis.

Aritméticos y comentarios

105 +, 106 -, 107 *, 108 /, 128 %, 129 ++, 130 -- y 133 **. Los estados 19 a 25 distinguen /, comentarios // y /* */. Los comentarios 131/132 se omiten del flujo sintáctico.

Delimitadores EOF y extensiones

Espacio, tabulador y salto de línea avanzan posición. EOF entrega una sola terminación; el marcador $ detiene la lectura aunque tenga texto después. MSeparadores añade : y .. sin cambiar M2; MLatin clasifica letras acentuadas en ids.

Las tablas heredadas M2[26][32], predictiveMatrix[49][60] y productions siguen intactas. Las matrices II agregan la semántica sin delegar el reconocimiento a bibliotecas de tokenización. Qt presenta las tablas y los controles.

CASE como extensión de Autómatas II

CASE no aparece entre los 29 diagramas sintácticos de la Fe de errata. Se conserva como ampliación del material II, páginas impresas 42 y 43. Sus acciones están enlazadas a las producciones 108 a 124 y al estatuto 107.

A3025 CASE_BEGIN Exigir selector ordinal int/char/bool; conservarlo apilado y crear marca y contexto CASE.

A3026 CASE_LOW POP límite inferior; guardarlo como ambos extremos provisionales.

A3027 CASE_HIGH POP límite superior y guardarlo en el contexto de rango.

A3028 CASE_TEST Validar tipos, orden y superposición; emitir igualdad o cierre de rango con <= y &&; reservar gotov y conservar selector.

A3029 CASE_LABELS_END Rellenar gotov de las etiquetas con CONT+1; emitir goto pendiente de no coincidencia y apilarlo.

A3030 CASE_ARM_END Emitir goto pendiente de salida; rellenar la no coincidencia hacia la próxima prueba o else.

A3031 CASE_END POP selector y tipo; rellenar salidas hasta la marca CASE; liberar temporal y cerrar contexto.

A3032 ORD_NEG Registrar el entero con signo negativo como constante y apilar AP y tipo.

A3042 CASE_RANGE_BEGIN Después de .., comparar selector >= inferior y reservar resultado antes de reconocer el superior.

En una lista se reservan varios gotov hacia el mismo cuerpo. Un rango genera >=, <= y &&. Al entrar al cuerpo se rellenan los verdaderos con CONT+1 porque en CONT se emite el goto de no coincidencia. Al terminar se rellenan las salidas de ramas hasta la marca de ese CASE.

La corrida del pizarrón conserva 18 cuádruplos. Destinos: Q6→8, Q7→10, Q9→18, Q13→15, Q14→17 y Q16→18. Se rechazan rangos invertidos y superpuestos; el selector y etiquetas deben tener el mismo tipo ordinal. Las etiquetas son literales, no expresiones generales.

Resolución de IF ELSEIF ELSE paso a paso

```
def a of int;
class
a=0;
if(a<0) write("neg");
elseif(a==0) write("cero");
else write("pos");
endif
endclass
```

1 DECL y ASSIGN

Declarar a; VALUE apila 0; STMT_NAME conserva a; ASSIGN emite Q1 = 0 → a.

2 IF_BEGIN

PUSH marca IF en PilaSaltos. No se reserva todavía una dirección falsa.

3 EXPR y OPREL

Apilar a, operador < y constante 0. Reducir: Q2 (<, a, 0, T1); operandos=[T1], tipos=[bool].

4 IF_TEST

POP T1/bool; emitir Q3 gotofalso T1 → pendiente. PilaSaltos=[marca, Q3].

5 Primer cuerpo

OUT emite Q4 para "neg". Al reconocer elseif, IF_NEXT emite Q5 goto pendiente y rellena Q3→6. PilaSaltos=[marca,Q5].

6 Segundo EXPR

Q6 (==, a, 0, T1). IF_TEST emite Q7 gotofalso pendiente. PilaSaltos=[marca,Q5,Q7].

7 Segundo cuerpo

Q8 out "cero". En else, IF_NEXT emite Q9 goto pendiente y rellena Q7→10. PilaSaltos=[marca,Q5,Q9].

8 ELSE e IF_END

Q10 out "pos". En endif, CONT=11: POP Q9 y Q5, rellenar ambos→11; POP marca. Pilas cerradas.

Con a=0 el recorrido de ejecución es Q1,Q2,Q3,Q6,Q7,Q8,Q9 y termina en Q11. No ejecuta los out de las otras ramas. El salto 11 es la salida, un destino válido aunque el último cuádruplo emitido sea Q10.

Funciones parámetros y activaciones

La Fe de errata define function id = TIPO (PARAMETROS) DECLARAVAR ESTATUTOS end_function. El material II explica ERA, PARAMETRO, GOSUB y RETORNO para procedimientos y pide extrapolarlo a funciones. La adaptación implementada añade destino de retorno y comprueba el tipo de la expresión retornada. La ampliación solicitada también acepta function id : TIPO (id : TIPO). Ambos separadores exigen un tipo explícito; no se infiere. La firma se muestra en la segunda vista de semántica.

```
def x of int;
function doble=int(x=int)
def y of int;
y=x*2;
return y;
end_function
class
x=7; write(doble(x),x);
endclass
```

Al definir doble, FUNC_NAME registra el módulo y un goto que salta su definición. El parámetro x y la local y reciben ámbito doble; el x global mantiene ámbito 0.

En la llamada, ERA salva locales y temporales del llamador en matrices de activación. PARAM copia el x global al espacio pendiente del argumento 1. Todavía no pisa un parámetro recursivo activo.

GOSUB guarda dirección de retorno y AP del temporal resultado. Inicializa los locales del destino, copia los argumentos por valor y transfiere el control a la entrada de doble.

La función calcula 14. RETURN guarda ese valor antes de restaurar las matrices del llamador. Luego deposita 14 en el temporal resultado y vuelve a la instrucción posterior al GOSUB.

WRITE consume el temporal y luego el x global, que sigue valiendo 7. Salida: 14 y 7, cada una en su línea.

La recursión usa el mismo descriptor de función y distintos registros de activación. FUNCIONES_AMBITOS.tls calcula factorial de 6 = 720 y conserva n global = 6. El límite comprobado es 64 activaciones.

Una función void se llama como estatuto y termina por endfunc. No puede aportar operando a una expresión. return EXPR es para no void; la gramática no incluye return sin expresión.

Las funciones deben estar declaradas antes del uso, salvo la recursión directa cuya firma ya se registró antes del cuerpo. No se añadieron declaraciones adelantadas, parámetros por referencia ni argumentos de tipo EXPR.

Matrices y comprobación reversible

MFunciones[64][8]

Tipo, entrada, primer parámetro, cantidad de parámetros, goto de definición, fin, primer símbolo local y cantidad de locales.

MParametros[256][3]

Función, símbolo destino y tipo. El orden de filas define el orden formal.

ambitos[256]

0 global; f+1 pertenece a la función f. Buscar local antes que global permite ocultamiento.

MLlamadas[64][3]

Contexto del generador: función llamada y siguiente argumento. Comprueba firma y cantidad.

MActivaciones[64][4]

Función, PC de retorno, AP de resultado y estado de activación.

MEstadoActivacion y MNumeroActivacion y MTextoActivacion

Copias matriciales del tipo, inicialización, número y texto de locales y temporales. Los globales no se restauran: sus cambios sobreviven a la llamada.

MArgumentos y MTextoArgumentos

Argumentos pendientes por valor para evitar pisar parámetros del llamador en recursión.

Comprobaciones que acompañan este manual

1. Decodificar las 133 posiciones de MProducciones y quitar las acciones recupera exactamente gramatica_II.tsv. 2. Las 130 producciones activas tienen un diagrama de origen o una extensión explícita. 3. Las 58 acciones activas tienen explicación y ejecución observada. 4. FIRST/FOLLOW y la tabla no tienen conflictos. 5. Los cuatro controles CTest conservan los 373 casos de I y verifican la semántica II.

El TSV adjunto incluye diagrama, página, producción, recorrido anotado, columna física de MProducciones, cantidad de símbolos anteriores, código y nombre de acción, efecto y ubicación del case en C++. La misma acción puede servir a varias reglas; un auxiliar estructural puede no tener acción propia.

Errores de módulos que deben reconocerse

S081 nombre de función inválido o directorio lleno; S082 uso incorrecto de llamada; S083 parámetro inválido; S084 cantidad de argumentos; S085 tipo de argumento; S086 return fuera de función o camino sin retorno; S087 retorno incompatible o void como valor. En ejecución: R020 límite de activaciones; R021 parámetro fuera de activación; R022 GOSUB inválido; R023 retorno sin activación; R024 terminación no void sin valor.

Este manual documenta el código ejecutable y sus decisiones explícitas. No sustituye una revisión con el profesor de convenciones que el material no fija, como RAM=1000+AP. No se han implementado arreglos dimensionados del capítulo posterior: no forman parte de los diagramas sintácticos de la Fe de errata.